



Tecnológico de Monterrey

Campus Santa Fe

Pokémon TCG Database

Course:

Software Construction and Decision Making

Students:

Jesús José Espinoza Torruco

Matriculate:

A01781963

Delivery Date:

30 de abril

1. User Layer

These tables manage all the elements related to a client. They track the player identity, lifetime statistics, and account data.

1.1 user

Stores the identity record for every registered player. This table functions as the main table for the whole diagram.

This table is important because it associates runs, statistics, and progress with a specific person.

Relations

- user → deck (1:N): a user may own multiple decks.
- user → collection (1:N): a user owns many cards.
- user → match as player1 and player2 (1:N): a user participates in many matches. •
- user → match as winner (1:N): a user can win many matches.
- user → tournament_entry (1:N): a user can register for many tournaments.
- user → pack_opening (1:N): a user can open many packs.
- user → rank (1:1): a user has one active rank record.
- user → friend as requester and receiver (1:N): a user can have many friend connections.
- user → trade as sender and receiver (1:N): a user can participate in many trades.

Table

user		
Field	Type	Purpose
id (PK)	integer	Unique identifier for the user.
username	varchar	Displayed name chosen by the player.
email	varchar	Used for login and account recovery.
password	varchar	Hashed credential for authentication.
credits	integer	In-game currency balance.
crystals	integer	In-game premium currency balance.

2. Card Layer

This table represents all the cards used in the game. Keeping all of the cards in one section helps manage the cards in an easier way.

2.1 card

Stores the definition of every card available in the game. When a player adds a card to their deck or collection, other tables reference this record by card.id.

Relations

- card → collection (1:N): many collection entries reference the same card definition.
- card → deck_card (1:N): many deck slots reference the same card definition.
- card → pack_card (1:N): many pack opening results reference a card.
- card → trade as card_offered and card_wanted (1:N): cards are referenced in trade offers.

Table

card		
Field	Type	Purpose
id (PK)	varchar	Unique string identifier.
name	varchar	Name of the card
type	varchar	'Pokemon', 'Trainer', or 'Energy'.
subtype	varchar	'Basic', 'Stage 1', 'Item', 'Supporter'
set_name	varchar	Name of the set this card belongs to.
rarity	varchar	'Common', 'Uncommon', 'Rare', 'Ultra Rare'.
hp	integer	Hit points; none for other cards.

3. Collection Layer

This layer tracks the collection of cards own by the players, connecting the user with the card table.

3.1 collection

Represents the binder of a player. Each row represents the amount of copies a player has of a specific card.

Relations

- collection → user (N:1): many collection entries belong to one user.
- collection → card (N:1): many collection entries reference the same card definition.

Table

collection		
Field	Type	Purpose
id (PK)	integer	Unique identifier.
user_id (FK)	integer	References user.id.
card_id (FK)	varchar	References card.id.
quantity	integer	Number of copies owned.

4. Deck Layer

These tables represent the deck building system of the game. Players assemble 60-card decks from their collection, and then they use them in matches and tournaments.

4.1 deck

Stores the decks that belong to a player. A deck must contain 60 cards and must be done in based on the rules before the player can use it.

Relations

- deck → user (N:1): many decks can belong to one user.
- deck → deck_card (1:N): a deck is composed of many card entries.
- deck → match as deck1 and deck2 (1:N): a deck can be used in many matches.
- deck → tournament_entry (1:N): a deck can be registered for many tournaments.

Table

deck		
Field	Type	Purpose
id (PK)	integer	Unique identifier for the deck.
user_id (FK)	integer	References user.id.
name	varchar	Player-assigned deck name.
format	varchar	'Standard', 'Expanded', or 'Unlimited'.

is_valid	boolean	True when the deck passes all rules.
----------	---------	--------------------------------------

4.2 deck_card

Represents each card's entry of a deck to avoid duplicates of the same card. The constraints ensure each card appears only once per deck row. The count is stored in quantity.

Relations

- deck_card → deck (N:1): many card entries belong to one deck.
- deck_card → card (N:1): many deck entries reference the same card definition.

Table

deck_card		
Field	Type	Purpose
id (PK)	integer	Unique identifier for this deck slot.
deck_id (FK)	integer	References deck.id.
card_id (FK)	varchar	References card.id.
quantity	integer	Copies of this card in the deck (1-4 for most; unlimited for Basic)

5. Match Layer

This table helps manage every game played in the system. It stores both participants and the outcome of the match.

5.1 match

Represents the game between two players. The ids of both players are stored directly, and the winner is saved once the match concludes. The table also tracks whether a game is active, completed or abandoned.

Relations

- match → user as player1 (N:1): many matches involve the same user as player 1.
- match → user as player2 (N:1): many matches involve the same user as player 2.
- match → deck as deck1 and deck2 (N:1): decks used by each player.
- match → user as winner (N:1): references the winning user after the game ends.
- match → tournament (N:1): a match may optionally belong to a tournament bracket.

Table

match		
Field	Type	Purpose
id (PK)	integer	Unique identifier for the match.
player1_id (FK)	integer	References user.id of the first player.
player2_id (FK)	integer	References user.id of the second player.
deck1_id (FK)	integer	References deck.id used by player 1.
deck2_id (FK)	integer	References deck.id used by player 2.
winner_id (FK)	integer	References user.id of the winner; null for draws.
format	varchar	'Standard', 'Expanded', 'Unlimited', or 'Theme'.
started_at	timestamp	When the match began.

6. Tournament Layer

These tables manage the organized events that can be done in the game. The players register their deck and compete a bracket format tournament.

6.1 tournament

Defines a tournament event including its format, bracket structure, and lifecycle status. Tournaments host multiple matches and can award ranking points and prizes to top finishers.

Relations

- tournament → tournament_entry (1:N): a tournament has many registered players.
- tournament → match (1:N): a tournament hosts many individual matches.

Table

tournament		
Field	Type	Purpose
id (PK)	integer	Unique identifier for the tournament.
name	varchar	Display name of the event.
format	varchar	'Standard', 'Expanded', or 'Unlimited'.
structure	varchar	'Swiss', 'Single Elim', or 'Double Elim'.

status	varchar	'upcoming', 'registration', 'in_progress', or 'completed'.
starts_at	timestamp	Scheduled start time.

6.2 tournament_entry

Tracks each of the players' registration in a tournament with their deck submitted. The constraints prevent double registrations from the same player. The placement field changes when the tournament ends and the final standings are given.

Relations

- tournament_entry → tournament (N:1): many entries belong to one tournament.
- tournament_entry → user (N:1): many entries reference the same user across events.
- tournament_entry → deck (N:1): the deck registered for this tournament.

Table

tournament_entry		
Field	Type	Purpose
id (PK)	integer	Unique identifier for this registration.
tournament_id (FK)	integer	References tournament.id.
user_id (FK)	integer	References user.id.
deck_id (FK)	integer	References deck.id.
placement	integer	Final standing (1 = winner); none until tournament ends.

7. Pack Layer

These tables track every pack opened by a player and the cards received from each opening.

7.1 pack_opening

Stores the high-level event of a player opening a booster pack from a specific expansion. This table is the parent record that ties a pack opening to a user and a set.

Relations

- pack_opening → user (N:1): many openings belong to one user.
- pack_opening → pack_card (1:N): one opening yields many card results.

Table

pack_opening		
Field	Type	Purpose
id (PK)	integer	Unique identifier for this opening event.
user_id (FK)	integer	References user.id.
set_name	varchar	The expansion set that was opened.
cards_got	integer	Total cards received (default 10).
opened_at	timestamp	When the pack was opened.

7.2 pack_card

Records each individual card received from a single pack opening.

Relations

- pack_card → pack_opening (N:1): many card results belong to one opening.
- pack_card → card (N:1): many results reference the same card definition.

Table

pack_card		
Field	Type	Purpose
id (PK)	integer	Unique identifier for this result row.
opening_id (FK)	integer	References pack_opening.id.
card_id (FK)	varchar	References card.id.
slot	integer	Position within the pack (1-10).

8. Rank Layer

This table manages the competitive ladder, tracking each player's current tier and performance statistics.

8.1 rank

Stores the current competitive standing of a player. The unique constraint ensures each player has exactly one rank record. Tier progression from Beginner to Master is determined by accumulated points earned through ranked match wins. Wins and losses are tracked here to display the player's season record on their profile.

Relations

- rank → user (1:1): one rank record per user.

Table

rank		
Field	Type	Purpose
id (PK)	integer	Unique identifier for this rank record.
user_id (FK)	integer	References user.id.
tier	varchar	'Beginner', 'Intermediate', 'Advanced', 'Expert', or 'Master'.
points	integer	Ranking points within the current tier.
wins	integer	Total ranked wins this season.
losses	integer	Total ranked losses this season.

9. Friend Layer

This table manages the friends system by tracking friendship connections between players.

9.1 friend

Records the friend request and its status between two users. The unique constraint prevents duplicate requests. Once accepted, both users can challenge each other to practice matches and see each other's profiles. The blocked status allows a user to prevent further contact from a specific player.

Relations

- friend → user as requester (N:1): the user who sent the request.
- friend → user as receiver (N:1): the user who received the request.

Table

friend		
Field	Type	Purpose
id (PK)	integer	Unique identifier for this friendship record.
requester_id (FK)	integer	References user.id of the requesting player.

receiver_id (FK)	integer	References user.id of the receiving player.
status	varchar	'pending', 'accepted', or 'blocked'.

10. Trade Layer

This table supports the card trading system, allowing players to exchange cards from their collections.

10.1 trade

Records a trade offer sent from one player to another. The sender proposes to give a card in exchange for a wanted card. Only valid cards in the catalog can be traded. The status field tracks whether the offer is still pending, has been accepted, or was rejected. When a trade is accepted, the system transfers one copy of each card between the two players' collection rows.

Relations

- trade → user as sender (N:1): many trades can be sent by one user.
- trade → user as receiver (N:1): many trades can be received by one user.
- trade → card as card_offered (N:1): many trades can offer the same card.
- trade → card as card_wanted (N:1): many trades can request the same card.

Table

trade		
Field	Type	Purpose
id (PK)	integer	Unique identifier for this trade offer.
sender_id (FK)	integer	References user.id of the player offering the trade.
receiver_id (FK)	integer	References user.id of the player receiving the offer.
card_offered (FK)	varchar	References card.id of the card being offered.
card_wanted (FK)	varchar	References card.id of the card being requested.
status	varchar	'pending', 'accepted', or 'rejected'.
created_at	timestamp	When the trade offer was sent.